



Minicurso de C

Aula 01: Entrada, saída e tipos de dados

1

Entrada e saída de dados



O comando printf

Utilizamos esse comando para a saída de um dado.

```
// formato do printf  
printf("string formatação", [var, var, var,...]);
```

A "string de formatação" é um texto que podemos escrever literalmente ou indicar locais para a inclusão de variáveis usando "%" e um qualificador que define o tipo e a formatação de saída de cada item.

```
printf("%d %d %d \n", 1 , 2 , 3 );  
para cada %d o printf coloca o valor do próximo literal ou variável
```



O comando printf

código	significado
%d	decimal
%f	ponto flutuante
%c	caracter simples
%s	cadeia de caracter
%e	notação científica
%o	octal
%u	decimal sem sinal
%x	hexadecimal
%ld	decimal longo
%lf	ponto flutuante longo



Exemplo: Uso do printf



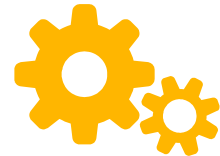
Os exemplos deste curso se encontram em um diretório do GitHub

[LINK PARA O GITHUB](#)



O comando printf

```
01. #include <stdio.h>
02.
03. int main()
04. {
05.     int dia = 0, mes = 0, ano = 0;
06.
07.     scanf("%d", &dia);
08.     scanf("%d", &mes);
09.     scanf("%d", &ano);
10.
11.     printf("%d/%d/%d\n", dia, mes, ano);
12.
13.     return 0;
14. }
```



Exemplo: Uso do scanf

[ACESSE AQUI](#)

Constantes e variáveis



Diferença entre elas

Constante: seu valor permanece inalterado durante a execução do programa. Sua declaração deve ser antecedida pela palavra reservada **const**.

Exemplo: `const int x = 10;`

Variável: seu valor pode variar durante a execução do programa

Variável local e global



```
#include<stdio.h>
#include<stdlib.h>
```

```
double PI = 3.141596;
```

← Variável Global

```
int soma(int valor1, int valor2){
    return valor1 + valor2;
}
```

← Parâmetros Locais

```
int main(){
    int resultado = soma(11, 49);
    printf("Soma = %d\n", resultado);

    system("Pause");
    return 0;
}
```

← Variável Local



Declaração de variáveis

Diferente do Python, em C devemos preceder a declaração de uma variável informando seu **tipo de dado**.

Exemplos:

- **int** a = 10;
- **float** b = 1.56;
- **char** c = 's';
- **void** d;

Tipos de dados primitivos



Numéricos inteiros

São representados por “**int**” e suas variações como signed/unsigned short/long/long long. Essas variações definem a capacidade de representação numérica.

Tipo	Capacidade
short int	16 bits (-32767 ... +32767)
unsigned short int	16 bits sem sinal (0 ... 65535)
int	16 bits (-32767 ... +32767)
unsigned int	16 bits sem sinal (0 ... 65535)
long int	32 bits (-2147483647 ... +2147483647)
unsigned long int	32 bits sem sinal (0 ... 4294967295)
long long int	64 bits (-9223372036854775807 ... +9223372036854775807)
unsigned long long int	64 bits sem sinal (0 ... 18446744073709551615)



Numéricos ponto flutuante

No C números de ponto flutuante pode ser representado por **float** ou **double** e suas variações.

Tipo	Capacidade
float	16 bits
double	32 bits
long double	64 bits



Caractere

No C caractere representa uma letra somente que é o tipo **char**. O char é um tipo escalar porque cada letra pode ser representada por um número como na **tabela ASCII**.

ASCII TABLE

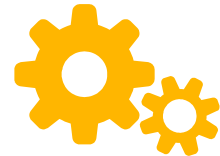
Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]



Nulo

O tipo void é um tipo de dados neutro que representa a ausência de valores. Podemos usar void como:

- **Retorno de função:** para indicar que a função não retorna nenhum valor.
- **Argumento de função:** Indicamos e reforçamos que a função não recebe parâmetros.
- **Ponteiros void:** Ponteiro neutro de tipo.



Exemplo: tamanho de variáveis

[ACESSE AQUI](#)



Obrigado!

Alguma dúvida?